



Compte Rendu – Projet Application de Jeu

- **Nom de l'établissement** : Faculté de sciences Semlalia
- **Filière / Département** : Informatique
- **Titre du projet** : Développement d'une application de jeu

Réalisé par :

- ORHOU RIDA : 2418404

- **Encadré par** : Prof. R. HANNANE
- **Année universitaire** : 2025–2026

Table des Matières

Compte Rendu – Projet Application de Jeu	1
Introduction.....	3
1. Spécification des besoins	3
1.1 Besoins fonctionnels	3
1.2 Besoins non fonctionnels	5
2. Architecture du projet	5
2.1 Fichier principal (main)	5
2.3 Fichier Game (coeur de l'application)	6
2.4 Fichier Joueur (Player).....	6
2.5 Obstacles	6
2.6 Coin.....	7
2.9 Fichier Resources (Assets).....	7
3. Structure des classes	8
3.1 Classe Game.....	8
3.2 Classe Obstacle / Coin	9
3.3 Classe Menu (Interface)	10
4. Relations entre les classes (POO)	10
4.1 Encapsulation	11
4.2 Héritage.....	12
4.4 Polymorphisme	13
4.5 Constructeurs et techniques utilisées	13
6. Environnement de travail.....	14
7. Interfaces du jeu.....	15
7.1 Interface principale	15
7.2 Interface de jeu.....	16
7.3 Interface de fin	17
8. Références.....	20
8.1. Documentation SFML	20
8.2. Cours de C++.....	20
9. Conclusion	21

Introduction :

Le présent rapport expose la conception ainsi que le développement du jeu *Forest Runner*, une application de jeu vidéo 2D à défilement horizontal réalisée dans le cadre du module de Programmation Orientée Objet en C++. Ce projet vise à mettre en pratique les principaux concepts de la POO notamment l'encapsulation, l'héritage, le polymorphisme et l'abstraction en les appliquant à un environnement de jeu en temps réel.

Le jeu s'appuie sur la bibliothèque graphique SFML 3.0.2, qui fournit une interface de haut niveau pour la gestion des éléments multimédias tels que les fenêtres 2D, les entrées clavier et souris, le rendu des textures et des sprites, ainsi que la lecture des effets sonores et de la musique. Ce choix permet de se concentrer davantage sur la conception orientée objet sans se soucier des aspects techniques bas niveau liés à l'API du système.

Dans *Forest Runner*, le joueur incarne un personnage devant traverser un environnement forestier hostile afin d'atteindre un refuge. Le déplacement est automatique, et le défi consiste à éviter des obstacles (comme des troncs d'arbres, des rochers ou des pièges naturels) générés aléatoirement, tout en collectant des pièces dispersées sur le parcours. Une limite de temps fixée à cinquante secondes impose une pression constante. La partie se termine en cas de collision avec un obstacle ou lorsque le temps est écoulé.

Enfin, ce rapport aborde successivement l'analyse des besoins fonctionnels et non fonctionnels, l'environnement de développement adopté, la description détaillée des classes C++ avec leurs attributs et méthodes, les relations entre ces classes illustrant les concepts de la POO, ainsi que les différentes interfaces graphiques du jeu accompagnées de leurs explications.

1. Spécification des besoins :

1.1 Besoins fonctionnels :

Les besoins fonctionnels définissent les fonctionnalités que le système doit assurer du point de vue de l'utilisateur. Ils sont organisés selon les principales fonctions du jeu *Forest Runner*.

Gestion du menu principal

- Le système doit afficher un menu principal animé dès le lancement de l'application.

- Le menu doit proposer au minimum trois options : Jouer, À propos et Quitter.
- La navigation dans le menu doit être possible au clavier (touches directionnelles) ainsi qu'à la souris (survol et clic), y compris en mode plein écran.
- L'option À propos doit ouvrir une fenêtre secondaire présentant les règles et les commandes du jeu *Forest Runner*.
- L'option Quitter doit permettre de fermer correctement l'application.

Déroulement d'une partie

- Le terrain doit défiler horizontalement de manière continue à une vitesse constante dès le début d'une partie.
- Le joueur peut faire sauter son personnage (touche Espace ou flèche Haut) afin d'éviter les obstacles au sol.
- Le joueur peut faire s'accroupir son personnage (flèche Bas maintenue) pour esquiver les obstacles en hauteur.
- Des obstacles de différents types (au moins trois) doivent apparaître de façon aléatoire avec une fréquence croissante au fil du temps.
- Des pièces à collecter doivent apparaître à intervalles réguliers, à des hauteurs variées, et être animées (oscillation verticale).
- Un compteur de pièces collectées doit être affiché en temps réel dans l'interface (HUD).
- Un compte à rebours décroissant doit être affiché ; lorsqu'il atteint zéro, la partie est perdue.
- Une barre de progression doit indiquer visuellement la distance parcourue vers l'objectif.
- Atteindre la distance de victoire avant la fin du temps déclenche l'écran de victoire.
- Tout contact avec un obstacle entraîne la mort du personnage, suivie d'une animation de transition puis de l'écran de fin de partie.

Gestion de l'audio

- Une musique de fond démarre dès l'affichage du menu principal et s'arrête à la fin de la partie.
- Un effet sonore est joué à chaque saut du personnage.
- Un effet sonore distinct est joué lors de la collecte d'une pièce.
- Un son spécifique est déclenché lors de la défaite.
- Une musique de victoire est jouée sur l'écran de victoire.

Écrans de fin de partie

- En cas de défaite, une image représentant le personnage mort doit s'afficher pendant deux

secondes avant l'écran de résultat.

- L'écran de fin de partie doit afficher le nombre total de pièces collectées.
- L'écran de victoire doit afficher le temps restant, le score obtenu ainsi qu'une évaluation des performances.
- L'utilisateur peut revenir au menu principal depuis n'importe quel écran de fin.

1.2 Besoins non fonctionnels :

Critère	Exigence
Performance	Le jeu doit maintenir 60 images par seconde de manière stable.
Portabilité	Le code doit compiler sous Windows 10 avec g++ (MinGW) et SFML 3.0.2.
Robustesse	Les ressources manquantes (textures, sons) sont signalées sans plantage.
Maintenabilité	Chaque classe est responsable d'un seul concept (SRP). Les constantes globales sont centralisées dans Constants.hpp.
Extensibilité	L'ajout d'un nouveau type d'entité nécessite uniquement l'héritage de la classe abstraite Entity et l'implémentation de son interface.
Ergonomie	Les commandes doivent être intuitives et fonctionner en fenêtré comme en plein écran.

Tableau 1 — Tableau des besoins non fonctionnels du projet Bunker Rush.

Ces contraintes non fonctionnelles guident les choix d'architecture : l'utilisation de `std::unique_ptr` pour la gestion mémoire garantit la robustesse, la séparation des constantes dans un fichier dédié facilite la maintenabilité, et l'interface abstraite Entity rend le système extensible sans modifier le code existant.

2. Architecture du projet :

L'application est organisée selon une architecture modulaire, où chaque fichier possède un rôle précis. Cette organisation permet de séparer les responsabilités et d'améliorer la lisibilité du projet.

2.1 Fichier principal (main) :

Le fichier principal contient la fonction main, qui sert de point d'entrée du programme. Il crée un objet Game puis lance le jeu avec la méthode `run()`. L'exécution est protégée par un bloc **try-catch** afin de gérer les erreurs. Si une exception se produit, un message d'erreur est

affiché et le programme se termine avec un échec. Sinon, il se ferme normalement après l'exécution du jeu.

2.2 Fichier Menu :

La classe **Menu** encapsule l'affichage et l'interaction du menu principal. Elle est entièrement auto-suffisante : elle charge sa propre police, gère sa propre horloge d'animation et communique avec **Game** via la valeur de retour entière de `handleEvent` (index de l'option choisie, ou valeur négative si aucune sélection n'est effectuée).

2.3 Fichier Game (cœur de l'application) :

Le fichier **Game** représente le composant central de l'application. Il est responsable de la gestion globale du jeu, notamment l'exécution, la logique interne, l'affichage graphique et la gestion des interactions utilisateur. Ce module orchestre l'ensemble des autres composants (joueur, obstacles, pièces, menu) afin d'assurer un fonctionnement cohérent et fluide.

Il gère également les différents états du jeu, tels que le menu principal, la phase de jeu, la défaite, et la victoire. Cette organisation permet de contrôler précisément le déroulement du jeu et d'adapter le comportement du système selon le contexte.

De plus, ce fichier assure la gestion des ressources (images, sons, textes), l'animation des éléments visuels, ainsi que le suivi des informations importantes comme le temps restant, la progression et le score. Il joue ainsi un rôle clé dans la coordination entre la logique du jeu et son rendu graphique.

2.4 Fichier Joueur (Player) :

La classe **Player** hérite publiquement de **Entity** et représente le personnage contrôlé par l'utilisateur. Elle gère sa propre physique (gravité, saut, accroupissement), ses trois textures animées et ses effets sonores.

2.5 Obstacles :

La classe **Obstacle** permet de créer et gérer les obstacles dans le jeu. Chaque obstacle est affiché avec une image et se déplace automatiquement vers la gauche à vitesse constante.

La méthode **update** gère son déplacement, et **draw** l'affiche à l'écran. La méthode **getBounds** sert aux collisions en définissant sa zone de contact. Enfin, **isOffScreen** vérifie si l'obstacle est sorti de l'écran pour le supprimer.

2.6 Coin :

La classe **Coin** hérite de **Entity** et représente une pièce que le joueur peut collecter pour augmenter son score. Son sprite référence la texture partagée stockée dans **Game**, garantissant la stabilité de l'adresse sans duplication de ressource

2.7 Fichier **Entity** :

La classe **Entity** sert de classe de base abstraite pour toutes les entités du jeu (joueur, obstacles, pièces, etc.). Son intérêt principal est de définir une interface commune que toutes les entités doivent respecter.

Elle impose certaines fonctions essentielles comme la mise à jour (**update**), l'affichage (**draw**), la gestion des collisions (**getBounds**) et la vérification si l'objet est hors écran (**isOffScreen**). Ainsi, chaque objet du jeu possède les mêmes fonctionnalités de base, mais avec son propre comportement.

Grâce à cette classe, le code devient plus structuré, réutilisable et extensible, car on peut manipuler toutes les entités de manière uniforme sans connaître leur type exact.

2.8 Fichier **constants** :

Le fichier **Constants.hpp** regroupe l'ensemble des valeurs fixes du jeu afin de simplifier la gestion et la modification du code. Il définit les dimensions de la fenêtre, les paramètres du monde (position du sol, départ du joueur, vitesse de défilement), ainsi que les règles de progression comme la durée de la partie et la distance de victoire. Il contient également les paramètres d'apparition des obstacles et des pièces, les ajustements des hitbox pour les collisions, ainsi que certaines constantes liées à l'interface utilisateur.

2.9 Fichier **Resources (Assets)** :

Ce module gère toutes les ressources du jeu (images, sons et polices). Il centralise leur chargement afin d'éviter les doublons et les erreurs. Les ressources sont chargées une seule fois

puis réutilisées dans tout le programme, ce qui rend le code plus propre, efficace et mieux organisé.

3. Structure des classes :

3.1 Classe Game :

La classe Game est le contrôleur central de l'application. Elle implémente la boucle de jeu, orchestre tous les autres objets et gère les transitions entre les cinq états de la machine à états finis.

Methodes utilisees :

- La fonction `Game()` (constructeur) initialise le jeu en créant la fenêtre, en définissant les variables principales comme le temps et le score, puis en appelant le chargement des ressources afin de préparer tous les éléments nécessaires dès le début.
- La fonction `loadResources()` permet de charger les images, les sons, les musiques et les textes utilisés dans le jeu, et de configurer les éléments graphiques pour qu'ils soient prêts à être affichés.
- La fonction `run()` contient la boucle principale du jeu, qui s'exécute en continu et enchaîne la gestion des événements, la mise à jour du jeu et l'affichage à chaque instant.
- La fonction `handleEvents()` gère les interactions du joueur avec le clavier, comme le saut, l'accroupissement ou le retour au menu, en fonction de l'état actuel du jeu.
- La fonction `resetGame()` réinitialise tous les éléments du jeu, notamment le score, le temps, les obstacles, les pièces et l'état du joueur, afin de recommencer une nouvelle partie.
- La fonction `update()` contient la logique principale du jeu en mettant à jour le temps, les positions, en générant les obstacles et les pièces, en détectant les collisions et en vérifiant les conditions de victoire ou de défaite.
- La fonction `render()` est responsable de l'affichage global du jeu en dessinant les éléments à l'écran selon l'état courant, comme le menu, le jeu ou les écrans de fin.
- La fonction `renderHUD()` affiche les informations importantes pour le joueur, comme le temps restant, le score et la progression.

- La fonction `showVictoryScreen()` affiche un écran de victoire avec les résultats et les performances du joueur.
- La fonction `showGameOverScreen()` affiche un écran de défaite indiquant que le joueur a perdu ainsi que son score.
- La fonction `showDyingScreen()` affiche une transition visuelle après la mort du joueur avec un effet sombre et une image spécifique.
- La fonction `showAboutScreen()` ouvre une fenêtre d'information qui explique le principe du jeu et les commandes.
- Les fonctions utilitaires `formatTime()` et `centerText()` servent respectivement à formater le temps sous forme lisible et à centrer les textes à l'écran.

Afin d'optimiser les performances du jeu et d'améliorer l'expérience utilisateur, plusieurs fonctionnalités supplémentaires ont été intégrées et perfectionnées :

- Mise en place d'un système de progression par niveaux (levels) avec augmentation de la difficulté en fonction de la distance parcourue.
- Mise en place d'une barre de niveau (**level bar**) affichant la progression vers le niveau suivant ainsi qu'un changement de couleur selon le niveau.
- Introduction d'un effet visuel lors du changement de niveau (flash "LEVEL X !" au centre de l'écran).
- Introduction d'une variable `distanceTraveled` pour mesurer la progression du joueur et déterminer les niveaux et la victoire.
- Mise en place d'une barre de progression vers la victoire indiquant l'avancement global dans la partie.
- Amélioration du système de génération avec :
 - `spawnObstacle()` : génération dynamique des obstacles avec une difficulté progressive.
 - `spawnCoin()` : apparition des pièces à différentes hauteurs.
- Introduction d'un nouvel état DYING pour gérer une transition visuelle entre le jeu et l'écran de défaite.
- Amélioration des écrans de fin :
 - écran de victoire avec animations, statistiques et évaluation (note),
 - écran de défaite avec affichage du score et animations.

- Intégration de sons et musiques (musique de fond, son de collecte, son de défaite, son de victoire).
- Mise en place d'un effet de défilement du background pour donner une impression de mouvement continu.

3.2 Classe Player :

La classe *Player* est une spécialisation de *Entity* (héritage public) et modélise le personnage manipulé par l'utilisateur.

Elle contient comme attributs privés : les textures associées aux différents états (course, saut et accroupissement), un sprite, une vitesse verticale, un état courant (au sol, en saut ou accroupi), ainsi que des effets sonores.

- **Constructeur** : il charge les différentes textures et positionne le sprite au point de départ défini dans *Constants.hpp*.
- **jump()** : permet au joueur de sauter en appliquant une vitesse verticale négative lorsqu'il est au sol.
- **crouch()** / **uncrouch()** : ajustent la hitbox et la texture en fonction de l'état du personnage.
- **update(float dt)** : gère l'effet de la gravité, met à jour la position verticale et vérifie le contact avec le sol.
- **draw()** / **getBounds()** / **isOffScreen()** : assurent l'implémentation des méthodes définies dans l'interface *Entity*.

3.3 Classe Obstacle / Coin :

Attributs :

Cette partie regroupe les entités du jeu : **obstacles et pièces (coins)**, mais elles sont implémentées dans deux classes séparées.

- **Obstacle** : position, type (ground / low / high), vitesse, sprite, texture, hitbox
- **Coin** : position, vitesse, sprite, texture, hitbox, animation (flottement)

Méthodes Obstacle :

- La fonction **Obstacle::Obstacle(float spawnX)** est le constructeur de la classe. Elle permet de créer un obstacle avec un type aléatoire (sol, bas ou haut). Elle charge la texture correspondante, configure le sprite (taille, origine et position) et place l'obstacle à une position donnée sur l'écran avec une vitesse de déplacement vers la gauche.

- La fonction **update(float dt)** met à jour la position de l'obstacle en le faisant avancer vers la gauche selon sa vitesse et le temps écoulé, ce qui crée l'effet de défilement du jeu.
- La fonction **draw(sf::RenderWindow& w)** affiche l'obstacle à l'écran en utilisant son sprite, uniquement s'il existe.
- La fonction **getBounds()** retourne la zone de collision (hitbox) de l'obstacle. Elle ajuste légèrement cette zone pour rendre les collisions plus précises et réalistes.
- La fonction **isOffScreen()** vérifie si l'obstacle est sorti de l'écran (à gauche). Dans ce cas, il peut être supprimé afin d'optimiser les performances du jeu.

Méthodes Coin :

- La fonction **Coin::Coin(const sf::Texture& tex, float x, float y)** est le constructeur. Elle crée une pièce, lui applique une texture, la redimensionne, la centre et la place à une position donnée.
- La fonction **update(float dt)** met à jour la pièce en la faisant avancer vers la gauche et en ajoutant un léger mouvement vertical pour un effet d'animation.
- La fonction **draw(sf::RenderWindow& w)** affiche la pièce à l'écran.
- La fonction **getBounds()** retourne sa zone de collision pour détecter la collecte.
- La fonction **isOffScreen()** vérifie si la pièce est sortie de l'écran pour la supprimer.

3.4 Classe Menu (Interface) :

Attributs :

La classe **Menu** contient les dimensions de la fenêtre, la police, le titre et les options du menu. Elle utilise une liste d'items et un index pour gérer la sélection, ainsi qu'une horloge pour les animations.

Méthodes utilisées :

- La fonction **Menu(float w, float h)** est le constructeur. Elle charge la police et crée les éléments du menu en les positionnant à l'écran.
- La fonction **handleEvent(sf::RenderWindow&, Event&)** gère les entrées clavier et souris pour naviguer dans le menu, en adaptant les coordonnées avec `mapPixelToCoords`.

- La fonction **draw(sf::RenderWindow&)** affiche le menu, incluant le fond, le titre animé et les options.
- La fonction **moveUp()** permet de sélectionner l'option précédente du menu.
- La fonction **moveDown()** permet de sélectionner l'option suivante.

4. Relations entre les classes (POO) :

4.1 Encapsulation :

L'encapsulation est globalement bien respectée dans le projet. Chaque classe (Obstacle, Coin, Menu, Game, Player) définit ses attributs en **private** et expose uniquement les méthodes nécessaires en **public**. Cela garantit la protection des données et une meilleure organisation du code.

Les ressources globales sont soit gérées par la classe Game, soit définies dans [Constants.hpp](#) sous forme de constantes, assurant une utilisation propre et sécurisée avec SFML.

Classe Obstacle :

La classe Obstacle représente les obstacles du jeu.

Attributs privés : texture, sprite, type, speed

- **Obstacle(float spawnX)** : initialise un obstacle avec un type aléatoire, charge la texture correspondante et positionne le sprite au niveau du sol. L'objet est géré via [unique_ptr](#), sans copie ni déplacement.
- **update(float dt)** : déplace l'obstacle vers la gauche à vitesse constante.
- **draw(sf::RenderWindow&)** : affiche l'obstacle s'il est actif.
- **getBounds()** : retourne une hitbox ajustée pour des collisions précises.
- **isOffScreen()** : permet de détecter si l'obstacle doit être supprimé.

Classe Coin :

La classe Coin représente les pièces collectables.

Attributs privés : sprite, speed, baseY, bobTimer

- **Coin(const sf::Texture&, float x, float y)** : initialise la pièce, ajuste sa taille et son positionnement.
- **update(float dt)** : déplace la pièce avec un effet de flottement vertical.
- **draw(sf::RenderWindow&)** : affiche la pièce.
- **getBounds()** : retourne la zone de collision.
- **isOffScreen()** : vérifie si la pièce est hors écran.

Classe Menu :

La classe Menu gère l'interface utilisateur principale.

Attributs privés : font, items, selectedIndex, titleText, subtitleText, animClock, width, height

- **Menu(float width, float height)** : charge la police et initialise les éléments du menu.
- **handleEvent(sf::RenderWindow&, sf::Event&)** : gère les entrées clavier et souris et retourne l'option sélectionnée.
- **moveUp()** / **moveDown()** : changent la sélection du menu.
- **draw(sf::RenderWindow&)** : affiche le menu avec animations et effets visuels.

Classe Game :

La classe Game est la classe principale qui contrôle l'ensemble du jeu.

Attributs principaux : fenêtre, état du jeu, ressources, HUD, joueur, obstacles, pièces et menu.

- **loadResources()** : charge toutes les ressources du jeu.
- **resetGame()** : réinitialise une partie.
- **handleEvents()** : gère les événements selon l'état du jeu.
- **update(float dt)** : met à jour tous les éléments du jeu.
- **render()** : gère l'affichage global.
- **spawnObstacle()** / **spawnCoin()** : génèrent les objets.
- **checkCollisions()** : détecte les collisions.

4.2 Héritage :

L'héritage est mis en place à travers la classe abstraite **Entity**. Les classes **Obstacle**, **Coin** et **Player** héritent de cette classe et redéfinissent ses méthodes virtuelles (**update**, **draw**, **getBounds** et **isOffScreen**). Cette organisation permet de traiter toutes les entités du jeu de manière uniforme et cohérente, tout en conservant un comportement spécifique pour chacune.

4.3 Abstraction :

La classe **Entity** est purement abstraite : elle ne peut pas être instanciée directement. Elle définit un contrat minimal que tout objet du monde de jeu doit honorer. Ce niveau d'abstraction permet à **Game** de raisonner en termes de « que faire avec une entité » (la mettre à jour, la dessiner, vérifier si elle est hors écran) sans s'occuper de « comment elle le fait ».

4.4 Polymorphisme :

Le polymorphisme est mis en œuvre grâce à la classe **Entity**. Les méthodes virtuelles permettent d'appeler automatiquement la bonne implémentation selon le type réel de l'objet. Cela facilite la gestion des entités dans la classe **Game** en offrant une structure plus flexible et extensible.

4.5 Constructeurs et techniques utilisées :

Les constructeurs servent à initialiser les objets avec leurs textures, positions et différents paramètres de départ. Le projet utilise **std::optional** pour sécuriser l'initialisation des sprites et **std::unique_ptr** pour assurer une bonne gestion de la mémoire. De plus, les classes **Obstacle** et **Coin** sont non copiables afin d'éviter toute duplication d'objets et garantir la stabilité du programme.

5. Méthodes SFML utilisée :

SFML (Simple and Fast Multimedia Library) est utilisée dans ce projet pour gérer tout ce qui concerne l'affichage graphique, les événements, le texte, le temps et les collisions.

Gestion des textures et sprites

- **sf::Texture::loadFromFile()** charge une image dans le jeu.
- **sf::Sprite(sf::Texture&)** permet d'afficher cette image.
- **setPosition(x, y)** place l'objet dans la fenêtre.

- `setScale(x, y)` change la taille du sprite.
- `setOrigin(x, y)` définit le centre de l'objet.
- `move(x, y)` déplace l'objet dans la scène.

Gestion de la fenêtre

- **`sf::RenderWindow`** : représente la fenêtre principale du jeu.
- **`draw(object)`** : affiche un élément graphique dans la fenêtre.
- **`clear()`** : efface l'écran à chaque frame.
- **`display()`** : affiche le contenu rendu à l'écran après chaque frame.

Gestion des événements

- **`sf::Event`** : permet de détecter les actions de l'utilisateur.
- **`KeyPressed`** : détecte une pression sur une touche du clavier.
- **`MouseMoved`** : détecte le mouvement de la souris.
- **`MouseButtonPressed`** : détecte un clic de souris.

Gestion du temps et des animations

- **`sf::Clock`** : mesure le temps écoulé dans le jeu.
- **`getElapsedTime().asSeconds()`** : retourne le temps en secondes.
- **`delta time (dt)`** : rend les mouvements indépendants du FPS.

Gestion des collisions

- **`sf::FloatRect`** : représente un rectangle de collision.
- **`getGlobalBounds()`** : retourne la zone occupée par un sprite.
- Permet de détecter les collisions : joueur / obstacle et joueur / coin.

Gestion du texte et de l'interface

- **`sf::Font::openFromFile()`** : charge une police de texte.
- **`sf::Text`** : affiche du texte (menu, titre...).
- **`setString()`** : définit le contenu du texte.
- **`setCharacterSize()`** : définit la taille du texte.

- `setFillColor()` : change la couleur du texte.
- `setStyle(sf::Text::Bold)` : applique un style (gras, italique...).

6. Environnement de travail :

Langage	C++
Bibliothèque graphique	SFML
Compilateur	<code>g++ src/*.cpp -I./SFML-3.0.2/include -L./SFML-3.0.2/lib -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio -o game.exe</code>
Exécution	<code>.\game.exe</code>
IDE / Éditeur	VS Code

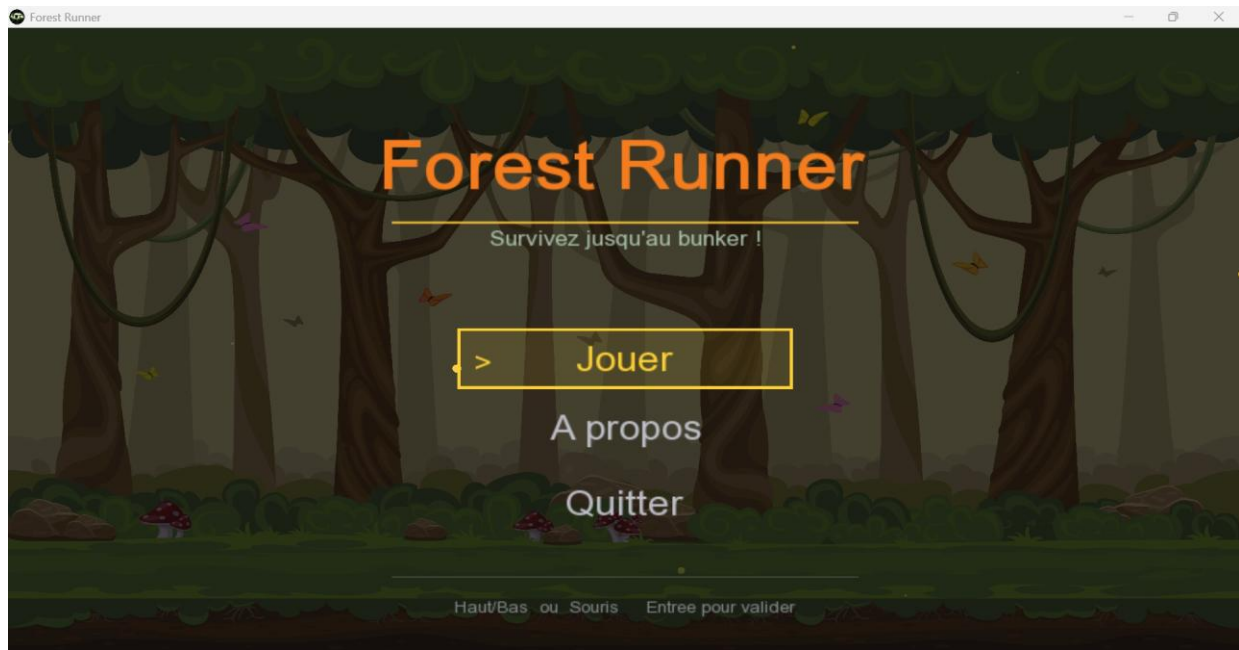
Ces outils permettent de développer une application graphique interactive de manière efficace.

7. Interfaces du jeu :

L'interface du jeu représente la partie visuelle avec laquelle le joueur interagit. Elle est créée grâce à SFML et permet d'afficher la fenêtre du jeu, les différents éléments graphiques comme le joueur, les obstacles, les pièces ainsi que le score. Elle rend le jeu plus clair, plus attractif et permet au joueur de comprendre facilement ce qui se passe en temps réel.

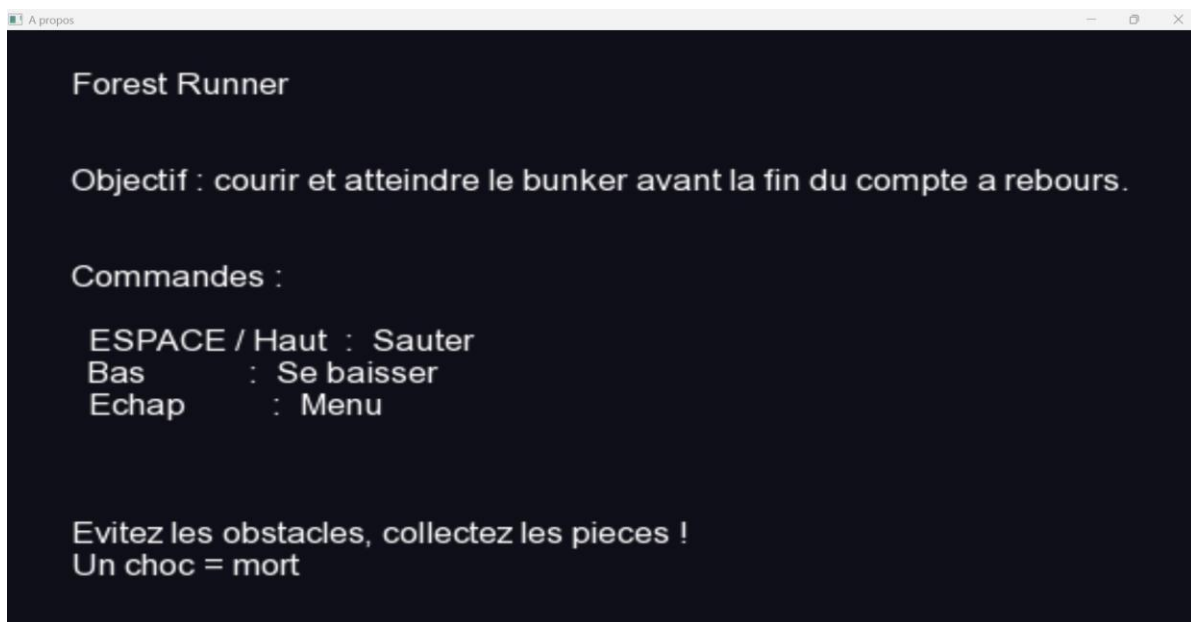
7.1 Interface principale :

- Menu avec options : jouer, à propos et quitter.
- Design simple et clair.



À propos :

Cette image présente l'écran 'À propos' du jeu Forest Runner, contenant l'objectif, les commandes et les règles du jeu.



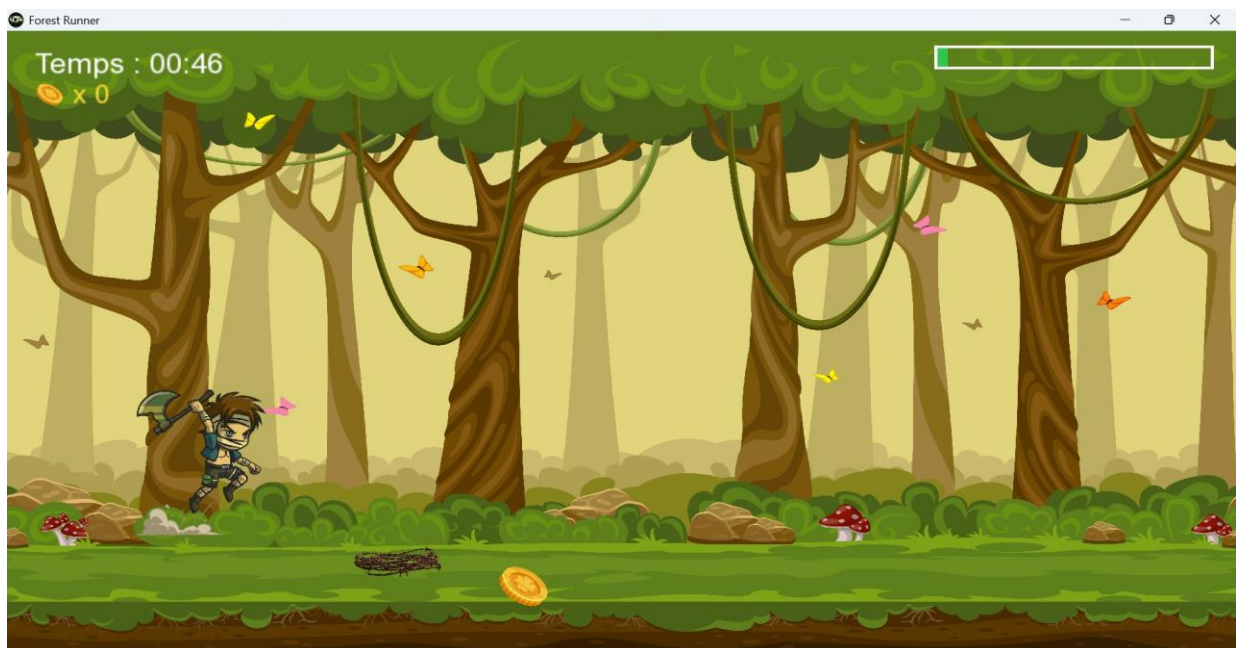
7.2 Interface de jeu :

Icône de l'application :



Cette image représente le logo du jeu Forest Runner.

Interface principale du jeu :



Cette image présente l'interface principale du jeu Forest Runner.

- Affichage du joueur et des obstacles.
- Visualisation du score.

- Interaction en temps réel.
- Barre de victoire du joueur

7.3 Interface de fin :

→ Affichage en cas d'échec du jeu :

Première page affichée :



Deuxième page affichée :



- Écran de game over affiché lorsque le joueur échoue
- Message de défaite (ex : “Game Over”)
- Cause de l’échec (collision ou temps écoulé)
- Affichage du score et du temps réalisé
- Option de rejouer ou retour au menu principal
- Indique la fin de la partie et incite à réessayer

→ **Affichage en cas de succès du jeu :**



- Écran de victoire affiché lorsque le joueur atteint l'objectif
- Message de félicitation ("Bunker atteint ! Bien joue !")
- Affichage du score et du temps réalisé
- Option de rejouer ou retour au menu principal
- Indique la fin du jeu et la réussite du joueur

Chaque interface est conçue de manière unique pour respecter les consignes du projet.

7.4 Personnage joueur :

Le joueur est représenté par un sprite de personnage militaire disponible en trois états visuels correspondant aux actions possibles en jeu.



7.5 Obstacles :

Trois types d'obstacles sont générés aléatoirement tout au long de la partie. Leur fréquence d'apparition augmente progressivement avec le temps écoulé, rendant la fin de partie plus difficile.



7.6 Pièce collectible :

Les pièces sont des objets dorés qui apparaissent toutes les 2 secondes à trois hauteurs (bas, milieu, haut). Elles bougent légèrement de haut en bas pour attirer l'attention. Quand le joueur les collecte, le score augmente et un son est joué.



8. Références :

Dans ce projet, nous avons utilisé plusieurs ressources pour mieux comprendre et réaliser le jeu en C++ avec SFML :

8.1. Documentation SFML :

La documentation officielle de SFML a été la principale référence. Elle nous a aidé à apprendre comment créer une fenêtre, afficher des images, gérer les événements (clavier, souris) et ajouter des sons.

8.2. Cours de C++ :

Les supports de cours en programmation C++ ont été utilisés pour comprendre la programmation orientée objet (classes, objets, constructeurs, méthodes) et organiser correctement le projet.

9. Conclusion :

Ce projet nous a permis de réaliser un jeu en C++ en utilisant la programmation orientée objet et la bibliothèque SFML. Il nous a aidés à mieux comprendre comment organiser un programme en plusieurs classes pour rendre le code plus clair et plus facile à gérer.

Grâce à la POO, nous avons pu séparer les différentes parties du jeu (joueur, obstacles, gestion du jeu), ce qui a amélioré la structure et la lisibilité du projet. De plus, SFML a permis de créer une interface graphique, d'afficher les éléments du jeu et de gérer les interactions avec l'utilisateur.

En résumé, ce projet a été une expérience utile et enrichissante qui nous a permis de développer nos compétences en programmation et de mieux comprendre le fonctionnement d'un jeu vidéo simple.